

UNIT V

File System: File System Interface: File concept, Access methods, Directory Structure; File system Implementation: File-system structure, File-system Operations, Directory implementation, Allocation method, Free space management; File-System Internals: File-System Mounting, Partitions and Mounting, File Sharing.

Protection: Goals of protection, Principles of protection, Protection Rings, Domain of protection, Access matrix.

Basic File Concepts

- Files are the building blocks of any operating system. Permanent storage of information and data is a file. File provides an easy means of information sharing. The operating system is not interested in the information stored in the files. Operating system map files with physical devices.
- An unstructured sequence of data means a file. It is also logical units of information created by processes. Information stored in the files must be persistent. File is any organized information and stored within the secondary storage. It is physical view of a file.
- A program which user prepare is a file. File represents programs and data. The output from compiler may be an object code file or an executable file.
- Content or information/data of file is decided by the creator of the file. File contains various types of data like source program, object program, text and numeric data, images, graphs, sounds, payroll records, dead-stock records etc.

File Naming

- File store information on the disk. It is an abstract mechanism. File name is given, when process creates a file. File name continues to exist even after closing a file. It is accessed by other process.
- File name rule is changed according the operating system. All operating system allowed upto 8 string as a file name. Special characters and numbers are allowed to use as a file name. Many file systems support names as long as 255 characters. File names contains two parts. It is separated by a period. For example: sorting.c
- In the file name, after period is called file extension. File extension indicates something about a file. UNIX operating system support more than one file extension. It is up to the user, for extending the file extension.
- MS-DOS support only one file extension. File extension after period contains upto three characters. Following is the list of file extension.

Sr. No.	File extension	Remarks
1.	.c	C source program file
2.	.exe	Executable file
3.	.bak	Backup file
4.	.hlp	Help file
5.	.pdf	Portable document format file
6.	.txt	General text file
7.	.zip	Compressed file
8.	.gif	Graphics interchange format image file
9.	.html	WWW html file
10.	.mp3	Music encoded in MPEG layer 3 audio file format
11.	.ps	Postscript file

Types of File

- File type represents the internal structure of the file. According to the structure, file is divided into certain types: text, source, executable and object file.

1. Text file: It is sequence of character which organized into lines. Text file is a regular file.

2. Source file: It contains sequence of subroutines, procedure and functions.

3. Executable file: It contains a series of code sections. This file is input to the loader and loader loads this file into memory and then execute.

4. Object file: Object file is input to the linker. An Object file is a binary file that the compiler creates that converts the source to object code.

File Types

File is divided into following types:

1. Ordinary file
2. Directory file
3. FIFO file
4. Character device file
5. Block device file.

Ordinary file

- Also called regular file. It contains only data as a stream of characters. An ordinary file cannot contain another file, or directory.
- It is either text file or a binary file.
- Examples of ordinary files include simple text files, application data files, files containing high-level source code, executable text files, and binary image files.
- **Text file:** It contains only printable characters.
- **Binary file:** It contains printable and unprintable characters that cover the entire ASCII range.
- Regular files may be created, browsed through, and modified by various means such as text editors.

Directory file

- A directory file is like a folder that contains other files, including subdirectory files.
- Directory files act as a container for other files, of any category. Directory files don't contain data in the user sense of data; they merely contain references to the files contained within them.
- Directory is created in UNIX by the *mkdir* command.
- The UNIX directory is considered empty if it contains no other files except the "." and "..".
- Directory is removed by using *rmdir* command. Content of directory is displayed by *ls* command.

Device file

- Special files are also known as device files. In UNIX all physical devices are accessed via device files; they are what programs use to communicate with hardware.
- Files hold information on location, type, and access mode for a specific device. There are two types of device files; character and block.
- **Block device** files are used to access block device I/O. Block devices do buffered I/O, meaning that the data is collected in a buffer until a full block can be transferred.
- **Character device** files are associated with character or raw device access. They are used for un-buffered data transfers to and from a device. Rather than transferring data in blocks the data is transferred character by character.
- One transfer can consist of multiple characters.

- Some devices, such as disk partitions, may be accessed in block or character mode. Because each device file corresponds to a single access mode, physical devices that have more than one access mode will have more than one device file.
- Device files are found in the `/dev` directory. Each device is assigned a major and minor device number."
- The major device number identifies the type of device, i.e. all SCSI devices, would have the same number as would all the keyboards.
- The minor device number identifies a specific device, i.e. the keyboard attached to workstation.
- Device files are created using the *mknod* command.
- Example of character device files is printers, modems and consoles.

FIFO file

- FIFO file is special pipe device file which provides temporary buffers for two or more processes to communicate by writing data to and reading data from the buffer.
- A FIFO is created by the *mknod* system call and may be opened and accessed by any process that knows the name and has the permissions.
- The buffer associated with a FIFO file is allocated when the first process opens the FIFO file for read or write. The buffer is discarded when all processes which are connected to the FIFO close their reference to the FIFO file.
- FIFO file may be removed like any regular file. FIFO files can be removed in UNIX via the `rm` command.

File Attributes

- One of the characteristics of file is a set of file attributes that give the operating system more information about the file and how it is intended to be used. For human users convenience bane is given to the file. File attributes are varies from system to system.
- File attributes are as follows:
 1. Name
 2. Identifier
 3. Type
 4. Location/place

5. Size

6. Protection

7. Date and Time

- File name is in human readable. User can perform any operation on file using its name.
- When file is created by user, operating system assigns unique identification number to each file. OS uses this identifier for its operation.
- Type information is required for systems that support different types of files.
- Location/place information is pointer to a device and actual location of files on the device. The information needed to locate the file on disk is kept in memory.
- Size is measured in bits or bytes. It gives idea about current size of the file.
- **Protection:** This information is stored on the per-process table so the operating system can allow or deny subsequent requests. Attributes like protection, password, creator and owner provides protection to a file.
- **Date and time:** This information is related to creation and modification of files. Protection, security and monitoring purposes this data is used by operating system.

Operation on File

- File operations are simply those things that user can perform on a file. For example, user can create a file, save a file, open a file, read a file and modify a file. There are many different types of file operations supported by operating system.
- Operating system can provides system call for performing various operations on the file. Six basic file operations are creating, write, read, delete, repositioning and truncating.
- Following are the some list of file operation:
 1. Create
 2. Write
 3. Closes
 4. Read
 5. Delete
 6. Repositioning.
- All these operations require that the directory structure be first searched for the target file.

- **File creation:** Any time user can create a file. File is also created without data. The steps for creating a file.

- a. Space: File system must provide a space for the file.

- b. New file entry is made in the directory.

- **Write:** To perform write operation on the file, file name and data is required. It updates a file by adding new data and changes some content of the file. For writing into a file, operating system searches the directory for particular file. Write pointer is kept at the location where the writing starts. Write pointer position is at middle of the file or end of the file. Write pointer is updated when writing to file is completed.

- **Close:** Process is not used a file after closing. Process can not perform any operation on file. Operating system remove it entry from open file table.

- **Read:** A process reads all or some portion of the data in a file. Read system call is used for reading a file. For reading a file, name and position of the memory is specified. Read pointer is kept into the file for next read operation. Read pointer is updated after completion of the read system call. Information about read pointer is stored in per process current file position pointer.

- **Delete:** Operating system remove the file from file structure. For deleting a file, no directory is searched with particular name and file is deleted. Deleting means making space free from memory and disk. Operating system also removes the directory entry. This free space is used by another file.

- **Repositioning:** The directory is searched for the proper entry, and the current-file-position pointer is repositioned to a given value. Repositioning within

- Other file operations are open, append, seek, get and set attributes and rename.

- **Open:** Before performing any operation on file, it is necessary to

- **Append:** Append is related to write operation of file. Append only add data to the end of the file.

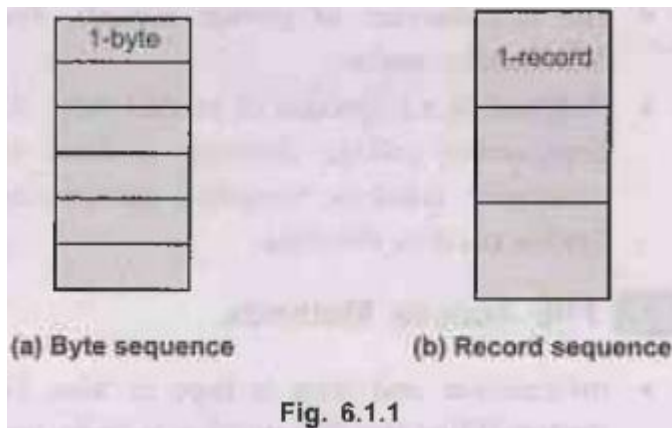
- **Get and set attributes:** Get attribute is used before performing operation on file. When user request a file for any operation, get attributes is executed by the process. Depending upon the attributes, user can allow or deny the operation. User can set the attributes and it changed after creation of file.

- **Rename:** User can change the name of existing file.

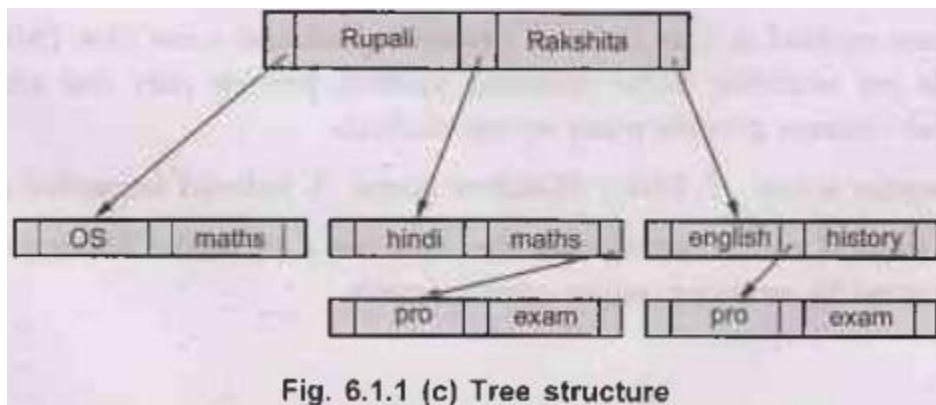
File Structure

- File structure is represented by field, record, file and database. It is also represented by byte sequence, record sequence and tree.

- Fig. 6.1.1 shows the three different kinds of files. Unix and windows both use unstructured byte sequence



- Byte sequence is unstructured file. Operating system is nothing to do with content of file. All content is in byte format. Windows and UNIX operating system uses this byte sequence method. It provides more flexibility to the user. User can put anything in the file and assign name whatever they want to the file.



- In record sequence model, file contains a sequence of fixed length record. Read and write operation on this type of file will perform one record at a time. It uses an idea of 80 column punched card. Most of the operating system based their file system on file consisting of 80 character records.
- Record sequence model also support 132 character records. Line printer uses 132 characters for printing.
- Last method is tree structure. File consists of tree of records. Each record contains key field in the fixed position. Key field is used to sort the tree record.
- Field, record, file and database is used in these file structure. Field is a basic element of data. Persons last name and date example of the field. It contains single value. Length and data type is characteristics of field.

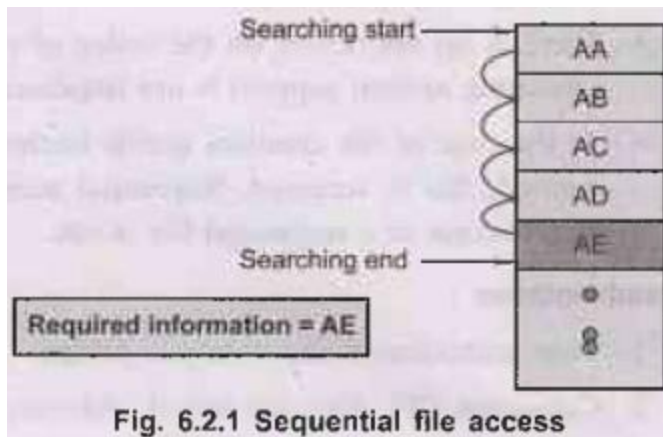
- Collection of related field is records. A student record contains name, address, mobile number, university exam number, college name etc. Records may be fixed length or variable length. A record also contains sub-records. Sub-records may be one or more than one.
- File is collection of similar records. From user view, file is single entity and reference by name.
- Database is a collection of related data. All records have some relation in database. Engineering college database contains information about college, student, staff, university, syllabus, timetable, fee' structure, various departments etc. One or more field is used in database.

File Access Methods

- Information and data is kept in files. Files are stored on the secondary storage device. When this information is to be used, it has to be accessed and brought into primary main memory.
 - Information in files could be accessed in many ways. It is usually dependent on an application. File organization checks how efficiently the input-output storage medium used.
 - File access method defines the way processes read and write files. Different access methods are available. Some operating systems provide only one access method and other systems provide many access methods.
1. Sequential access 2. Direct /Random access 3. Indexed sequential access.
- Access method means accesses to files that use a particular file organization are implemented by an input output control system.

Sequential Access Method

- Sequential access method simple method. The information in a file is accessed quentially one record after another.
- In this method, a process could read all the records in a file in order, starting at the beginning. It can not skip any records and can not read out of order.
- Fig. 6.2.1 shows sequential access method.



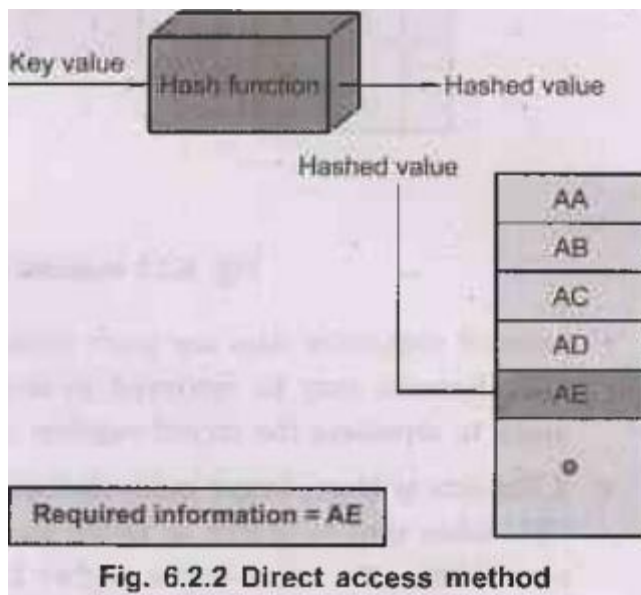
- Batch application uses sequential files. A byte stream file uses sequential access method.
- **Example:** Compiler and editor usually access files in this method. Transaction file is also example of sequential access method.
- Sequential file organization only method easily stored on tape and hard disk. Sequential access is best suited where most of the records in a file are to be processed.

Disadvantages:

- It provides poor performances.
- More efficient search technique is required.

Direct Access Method

- It is also called random access method. This access allows a user to position the read/write mark before reading or writing.
- Fig. 6.2.2 shows direct method.
- Direct access file method provides, accessing the records directly. Direct access method is based on the hard disk that is a direct access device. It allows random access of any file block.



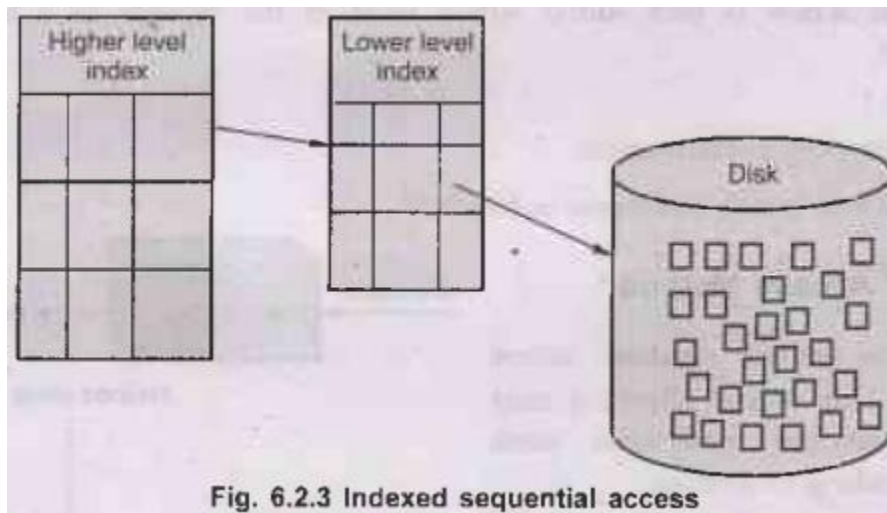
- Each record has its own address on the file with by the help of which it can be directly accessed for reading or writing. This feature is used by editors. An editors need to randomly access the contents of the file.
- There is no restriction on the order of reading or writing for a direct access file. Operating system support is not required for direct access file.
- At the time of file creation, access method is defined. According to defined access method, file is accessed. Sequential access of a direct access file is possible but direct access of a sequential file is not.

Disadvantages:

1. Poor utilization of input-output device.
2. Consumes CPU time for record address calculation.

Indexed Sequential Access

- It is modified method of direct access method. This method is combination of direct and sequential access method.
- In simple indexed sequential method, simple single level of indexing is used. Sequential file is used as index.
- Fig. 6.2.3 shows indexed sequential file. Indexed sequential access contains key field and a pointer to the main file.



- Indexed sequential files store records in the order that they are written to the disk. Records may be retrieved in sequential order or in random order using an index to represent the record number in the file.
- If file size is large, larger index is required and it contains large number of entries. CPU takes time to search in to the index. Higher level index is used to reduce the search time. An entry in the higher level index points to a section of the index. This higher index contains the records. This index is searched to find the section of the disk that may contain a required record.
- File system maintains an overflow file. It adds new records to an overflow file. The record in the main file that immediately precedes the new record in logical sequence is updated to contain pointer to the new record in the overflow file.
- The overflow file is merged with the main file during a batch update. The multiple indexes for the same key field can be set up to increase efficiency.
- The lowest level of index file is treated as a sequential file and a higher level index file is created for that file. Higher index file would contain pointers to lower index files, which would point to the actual data items.

File Directory and Directory Structure

- User stores file in file system using directory. File system stores files of many users. Files are stored on the secondary storage device like hard disk, optical disk and memory disk. These devices support random access method.
- Directory structure organizes and provides information about all files in the system. Every partition has a file system which consists of files and directory. Hard disks are split into one or more partitions. It is called as minidisks or volumes. User can assign names to volumes. Each disk contains minimum one partition.

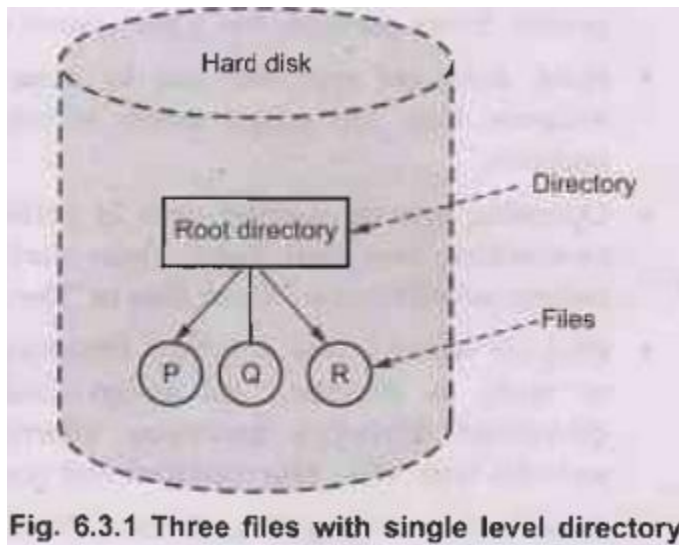
- Operating systems support upto 24 partitions. Partitions can be larger than a disk i.e combining two hard disks. These partitions are called "virtual disks". Partition maintains information about files in "Device Directory".
- Files are stored in the directory. Directory is created by user. User is free to create as many as directory and assign suitable names. File system contains many directories. Directory maintains information about the file. File information includes type, size, date modified and name.
- In some operating system, directory itself is considered as a file. A set of logically associated files is called directory. Directory structure of the file system connects a directory to other directories in the system.
- Directory is a data structure that lists files and subdirectories in a collection. Directory structure contains a list of entries one for each file.
- Directory does not store any user data. Single dot (.) indicates a current directory rote and double dot (..) indicates parent directory. The root is identified by the directory '/'. The root file system has several subdirectories.
- Windows file system uses a letter followed by colon for specifying root directory. For example, root directory in windows is C: or D: or E:
- UNIX and Linux based file system uses slash (/) for specifying root directory.
- A directory can contain any number of files. A directory can also contain subdirectories. Because a directory is nothing more than a special kind of file, directories also have names.

Operation on directory :

- 1. File searching:** Directory structure is searched for particular file entry. File uses symbolic names and similar names may indicate a relationship between files.
- 2. Create a file:** User creates new files and added to the directory.
- 3. Delete a file:** User remove the file after completing its work on files.
- 4. Rename a file:** User change the file name if file content is change.
- 5. Listing of directory:** Listing of files in the directory for some use. MS-DOS and windows uses "dir" command and Linux/UNIX uses "ls" command for this purposes.

Single Level Directory

- Single level directory is a simple structure and there are no sub-directories.
- Fig. 6.3.1 shows three files with single level directory. It is also called flat directory structure.



- Single level directory structure is suitable only for single user. If user increases, it creates the problem with assigning the name to files.
- In single level directory structure, no two files can have the same name.
- It is simple to implement and searching of file is faster. This type of directory system is used in cameras and phones.
- Because of limitation of file names, this structure is rarely implemented.
- File name size varies according to the operating system. MS-DOS operating system allows only 11 character file names and UNIX OS allows upto 255 characters.

Disadvantages of single level directory:

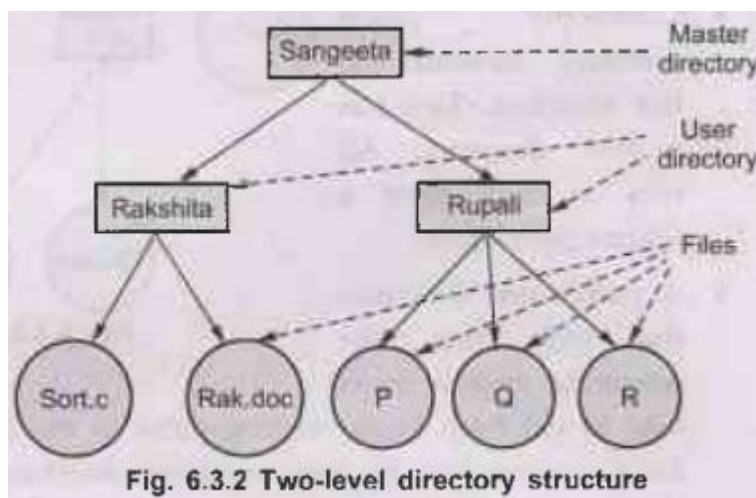
1. Not suitable for a large number of files
2. Limitation of unique file name.
3. For large number of file, recalling file name is difficult.

Hierarchical Directory System

- Here files are grouped together to form a sub directory. Each directory may contain several subdirectories. In hierarchical file system, a root directory points to other directories and files.
- Hierarchical directory system contains various forms of directory structures. They are as follows:
 1. Two-level directory structure
 2. Tree structured directories
 3. Acyclic graph directories.

Two-level Directory Structure

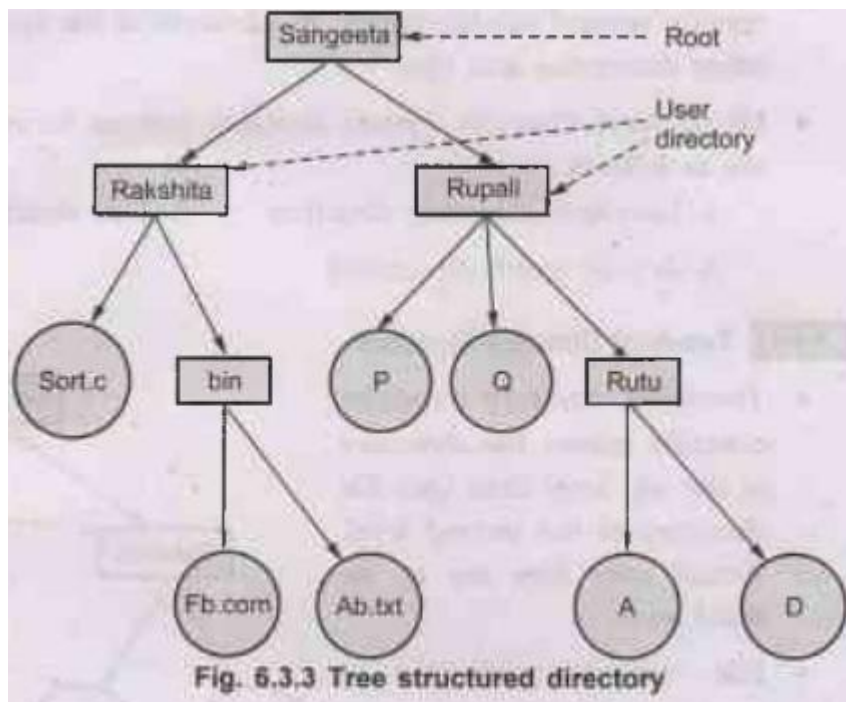
- Two-level directory structure contains master file directory at the top level then user file directory at the second level. Actual user files are at the third level.
- File system maintains a master block for each user. Master block has one entry for each user.
- User directory address is stored in the master block. Each user has a private directory. Different user can assign same names to their files as the directories for each user are now separate.
- Fig. 6.3.2 shows two-level directory structure.



- Here user can work only in own directory. Other user need not worry about deleting or modifying files by other users.
- When user wants to create a file, system search that particular file name in the directory. If same file name is not found, then it creates a file otherwise it gives error messages.
- In this structure, different users may have files with same name. It solves the Viol name conflict problem. When user want to open a file, filename is searched for in users directory.
- A two level directory can be a tree or an inverted tree of height 2.
- There are still problems with two-level directory structure. It effectively isolates one user from another. This is some time advantage or some time disadvantages. For completely independent user is an advantages and disadvantages when cooperation between two users is required.

Tree Structured Directory

- Fig. 6.3.3 shows the tree structured directory.



- In this structure, directory itself is a file. A directory and sub directory contains a set of files. Internal format is same for all directories
- Commonly used directory structure is tree structure. Tree has a root directory. All files in disk have a unique path name.
- A path name describes the path that the operating system must take to get from some starting point in the file system to the destination object. Each process has its own working directory. Path names are of two types : relative and absolute.
- Relative path name : Its path starts from current working directory (may start with
- Absolute path name: Its path starts from root directory (starts from "/").
- Tree structured directory differentiate file and subdirectory by using one bit representation. Zero (0) represents file and one (1) represents subdirectory.
- When process makes reference to a file for opening, file system search this file in the current directory. If needed file is not found in the current directory, then user either change the directory or give the full path of the file. System calls are used for performing any operation on the files or directory.
- Empty directory and its entry are easily deleted by operating system. If directory contains files and subdirectory, i.e. non-empty directory, some operating systems not delete the directory.
- To delete a directory, user must first delete all the files and subdirectory in that directory.

Merits

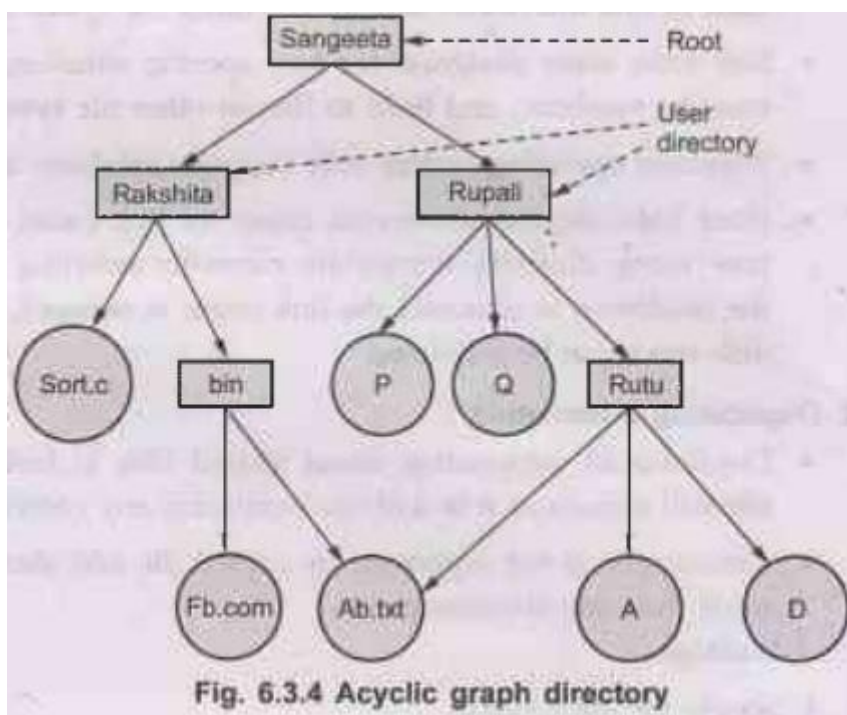
1. User can create his/her own directory.
2. User can access other user files by specifying path name.
3. Separate subdirectory for files associated with different topics
4. Searching is fast and efficient.

Demerits

1. Sub directory can not share between the user
2. Path is longer than two level directory structure

Acyclic Graph Directory

- Acyclic graph directory solve the problem of tree structure directory.
- It allows sharing the directory in between two users. At a time more than one places shared directory or file will exist in the file system.
- Fig. 6.3.4 shows acyclic graph directory. Links can be used to construct acyclic graph

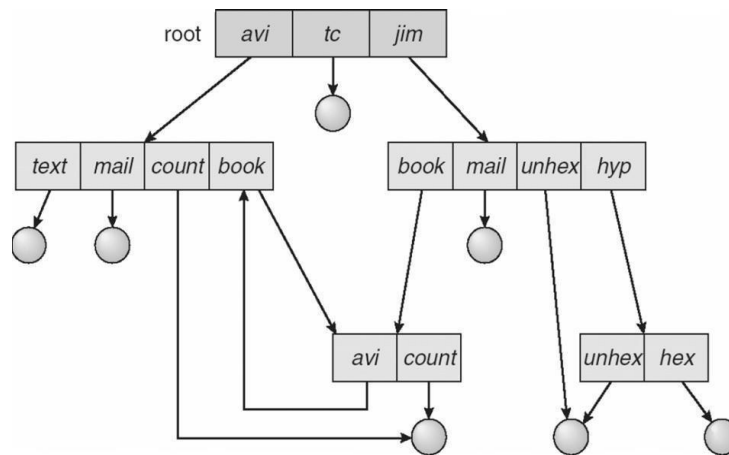


- It is very interesting to note that a shared directory or file is not the same as two copies of the file. When there are two copies of files, each user can view the copy rather than the original, but if one user changes the file content, the changes will be file content, it not appear in the other's copy.
- Only one original file exists for shared copy. Any changes made by one user are immediately visible to the other user. When user create file in shared directory, it automatically appear in all the shared subdirectories.

General Graph Directories

Adding new files and subdirectories to an existing tree-structured directory preserves the tree-structured nature. However, when we add links to an existing tree-structured directory, the tree structure is destroyed, resulting in a simple graph structure (Figure)

Figure: General graph directory.



In this case, we generally need to use a garbage-collection scheme to determine when the last reference has been deleted and the disk space can be reallocated.

Garbage collection involves traversing the entire file system, marking everything that can be accessed. Then, a second pass collects everything that is not marked onto a list of free space. Garbage collection for a disk-based file system, however, is extremely time consuming and is thus seldom attempted. Garbage collection is necessary only because of possible cycles in the graph.

A simpler algorithm in the special case of directories and links is to bypass links during directory traversal. Cycles are avoided, and no extra overhead is incurred.

Implementation of shared files and directory:

1. Using link :

- Link is used for creating shared files and directory. A link is a pointer to another file or subdirectory. If reference to a file is made, the directory is first searched. A link is a logical relationship between an inode and a file that relates the name of a file to its physical location.

- UNIX operating system uses acyclic graph directory structure. Reference count is maintained by UNIX for files and directory. UNIX operating system provides two types of links: hard link and symbolic link
- A hard link contains multiple directory entries that both refer to the same file. Hard links are only used with ordinary files in the same file system.
- A symbolic link is also called soft link. It contains a special file which maintains the information about file where to find the linked file. Symbolic links may be used to link directories and files in other file systems.
- Soft links were designed for two specific situations: Links to directories, which must be symbolic, and links to files in other file systems.
- Windows operating system only supports symbolic links.
- Hard links require a reference count, or link count for each file, keeping track of how many directory entries are currently referring to this file. Whenever one of the references is removed the link count is reduced, and when it reaches zero, the disk space can be reclaimed.

2. Duplicating information:

- Duplicate all information about shared files in both sharing directories. Original file will remain as it is without modifying any content by shared user.
- Consistency is not supported by copied file and shared file. Shared files may have more than one absolute path.

Advantages:

1. Simple for file traverse.
2. Structure is more flexible than a simple tree structure.
3. It allows directories and files to be shared between other users.

Disadvantages:

1. Deletion of file is difficult.
2. Problem with duplicate directory entries is maintaining consistency if the file is modified.

File System Implementation

- File system is implemented on disk and memory. How to implement the file system, it varies according to operating system and file system. But all the operating system follow some general rules. If the file system is implemented on the disk, it contains the following information:

1. Boot block control: Boot block is maintained per volume. The boot block contains solve the initial bootstrap program used to load the operating system. Operating system requires some information at the time of booting. If the disk is divided into number of partitions, the operating system is stored in the first partition of the disk. If the operating system is not installed on the disk, then this block can be empty. In UNIX file system, this is called the book block and in

2. Volume control block: It consists of volume or partitions detail information. The information like block size, number of blocks in the partition, free block count, free block pointer, free FCB count and FCB pointers. In UNIX operating system, each partition is a standalone file system. Super block is name in UNIX file system. A super block describes the state of the file system: The total size of the partition, the block size, pointers to a list of free blocks, the inode number of the root directory, magic number, etc. In network file system, it is stored in the master file table.

3. Directory structure: It is used to organize the files. Directory structure is maintained per file system.

4. Per-file PCB: It contains information about files such as file size, file ownership, file permission and location of data blocks. In NTFS, master file table stored this information. Master table file uses a relational database structure.

- In-memory information is used for caching and files system management. Caching improve the performance.

1. In-memory mount table: It contains information about each mounted volume.

2. In-memory directory structure cache: It contains recently accessed directories information.

3. System wide open table: Open files FCB information is stored.

4. Per-process open file table: It maintains the pointer to the appropriate entry in the system wide open file tables.

- UNIX operating system treats a directory exactly as a file. Windows NT implements separate system calls for files and directories. It treats directory as entities separate from files. Fig. 6.8.1 shows a file control block.

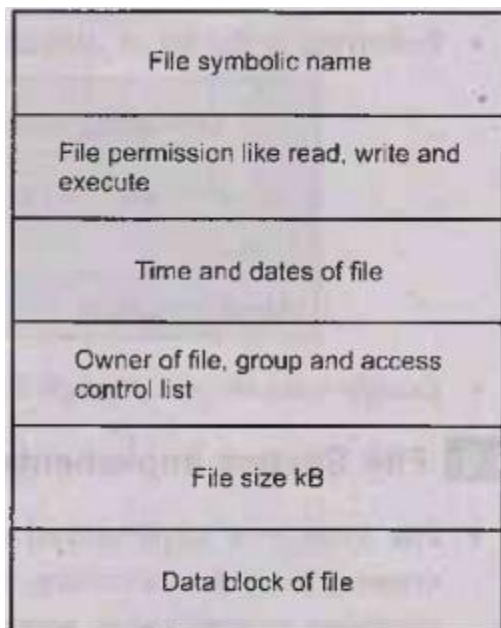


Fig. 6.8.1 File control block

- FCB specify the information that the system needs to manage a file. Sometimes it is called file attributes. These are highly system dependent structures. For creating new file, an application program calls the logical file system.

Optimizations for file system:

- Delayed updates of data and metadata are main difficulty. Updates could be sins delayed in the hope that the same data might be updated in the future or that the updated data might be temporary and might be deleted in the near future.
- If computer were to crash without having committed the delayed updates, then the consistency of the file system is destroyed.

Metadata updates:

- For recoverable file system, after crash, it must be consistent or must be able to be made consistent. So it is necessary to prove that logging metadata updates keeps the file system in a consistent.
- For inconsistent file system, the metadata must be written incompletely. Sometime, file system data structures is in wrong order. With metadata logging, the writes are made to a sequential log.
- The complete transaction is written there before it is moved to the file system structures. While updating data, file system crashed, the updates can be completed based on the information in the log.
- The logging ensures that file system changes are made completely. The order of the changes is guaranteed to be correct because of the sequential writes to the log. If a change

was made incompletely to the log, it is discarded, with no changes made to the file system structures.

File System Structure

- File system is mounted and maintain by the secondary storage which provides by disk. Characteristics of disk are as follows:
 1. Same place is used for reading, writing and modification.
 2. User can access disk directly for any block of information.
- Block is used for data transfer. Each block contains one or more sectors. The I/O transfer between memory and disk are performed on block. Default sector size is 512 bytes.
- Concept of file system is used for the efficient and convenient access to the disk. File system allows data to be stored, located and retrieved easily. The file system is generally composed of many different levels.
- Fig. 6.7.1 shows layers of file system.

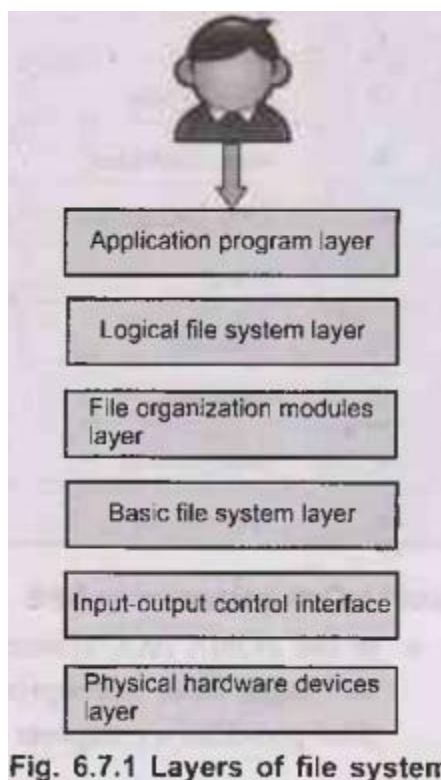


Fig. 6.7.1 Layers of file system

- There are six layers. Each layers give support to neighbour layers. It also provides function to above layer and below layer. Each layer uses the features of neighbour layer while creating new features for use by the higher levels.
- File system creates two main design problems:

1. Creation of algorithm and data structure for mapping of logical file system to physical secondary storage devices.

2. File system view for user.

- **Input-output control interface:** It consists of device driver and interrupt handler. Both are used for data transfer between main memory and disk system. Device driver used as a translator. Device driver input is high level commands and output is low level hardware specific instructions.

- **Basic file system layer:** It generates the command for device drivers. Particular device driver read data and writes physical blocks on the disk. Basic file system also manages the memory buffer and caches that hold various file system. Cache memory is used to hold meta-data information of file system.

- **File organization modules layer:** This layer maintain information about files and be their logical blocks and physical blocks. It translate the logical block address to physical block address by considering physical location of files and file allocation method. File logical numbers are do not match with physical block containing data, so translation is required. Free space manager is included in file organization modules.

- **Logical file system layer:** It manages metadata information. Actual data is not part of metadata information. But file system structure is part of metadata. Directory structure is used to organized file module. Operating system maintains file control block for each file. UNIX uses inode instead of file control block. File control block stored information about file, location, ownership and access permissions.

- **Application program layer:** User/programmer creates an application programs.

- **Physical hardware devices layer:** It contains actual hardware device link disk, memory etc.

- Following is the list of different operating systems file system:

Operating system	File system formats
Windows XP, NT and 2000	FAT, FAT32, NTFS
UNIX	UNIX File System (UFS)
Standard LINUX	Extended file system

Google uses its own Google file system.

Directory Implementation

- Directory is implemented by using linear list and hash table.

Linear list

- Linear list is simple method of directory implementation. It uses file names with bead pointers to data block. It is time consuming because of searching overheads.
- To create a new file, file name is searched in the directory to avoid the file name duplication. Then it adds new entry at the end of the directory. To delete a file, again file name is searched in the directory. Space is released after deleting a file. Directory entry is also removed. Unused space is marked as free entry. Last entry of directory is copied into the free space list.
- For deleting a file, some OS uses linked list. Linked list decreases time required.
- Searching the directory entry is the major disadvantage. It is slow process. Because of this reason, many operating system uses software cache memory. It stores the most recently used directory information.

Hash table

- Hash table is one more data structure used for directory implementation. Hash table takes a value computed from the file name and returns a pointer to the file name in the liner list.
- Hash table reduces the searching time. It uses fixed size block.

Allocation Methods

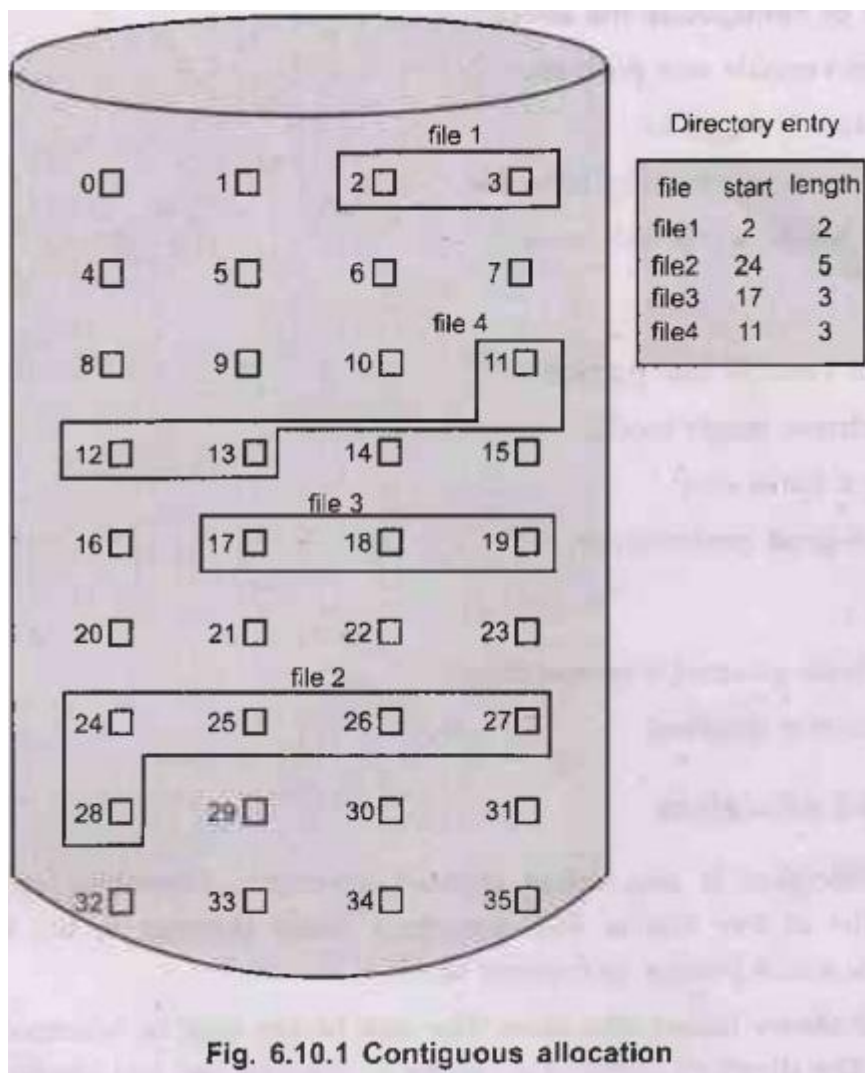
- The primary issue of file implementing file storage is how to keep track of file blocks. File consists of a collection of blocks.

File allocation

- Following issue are considered for file allocation:
 1. After creation of new file, whether the required maximum space for the file is allocated at once or not?
 2. Portion Space is allocated to a file as one or more contiguous units.
 3. Data structure or table used to keep track of the portion which is assigned to a file.
- Three major methods of allocating disk space are in wide use,
 1. Contiguous
 2. Linked
 3. Indexed.

Contiguous Allocation

- When user creates a file, system allocates a set of contiguous blocks on disk. This is a pre-allocation method that uses portion of variable size. Contiguous allocation is simple method to implement. It only search free list of correct number of consecutive blocks and mark them as used block.
- Disk address is a linear ordering on the disk. Because of linear ordering, accessing block $b + 1$ after block b normally requires no head movement. Here head movement is only one track. Fig. 6.10.1 shows contiguous allocation.
- Contiguous allocation of a file is defined by the disk address and the length of the on first block.
- If the file size is " n " blocks and starting location is " L ", then it occupies blocks $L, L+1, L+2, L+3, \dots, L+(n-1)$. The directory entry for each file indicates the address of the starting block and the length of the area allocated for this file.



- Sequential and random access can be supported by contiguous allocation.

- It is easy to retrieve single block. To improve the I/O performance of sequential processing, multiple blocks can be brought in one at a time. It supports sequential access very well because files entire data is stored in adjacent block. This method also supports random access.
- Contiguous allocation also suffers from external fragmentation. Small free disk spaces are created after allocation free block and deleting files.. External fragmentation means there will require free space available but that is not contiguous. To solve the problem of external fragmentation, compaction method is used.
- One more problem is that how to calculate the space needed for a file. It is difficult to estimate the required space.
- This method is good for storing data on CD-ROM.

Characteristic of contiguous file allocation :

1. It supports variable size portions.
2. Pre-allocation is required.
3. It requires only single entry for a file.
4. Allocation frequency is only once.

Advantages:

1. It supports variable size portion.
2. Easy to retrieve single block.
3. Accessing a file is easy.
4. It provides good performance.

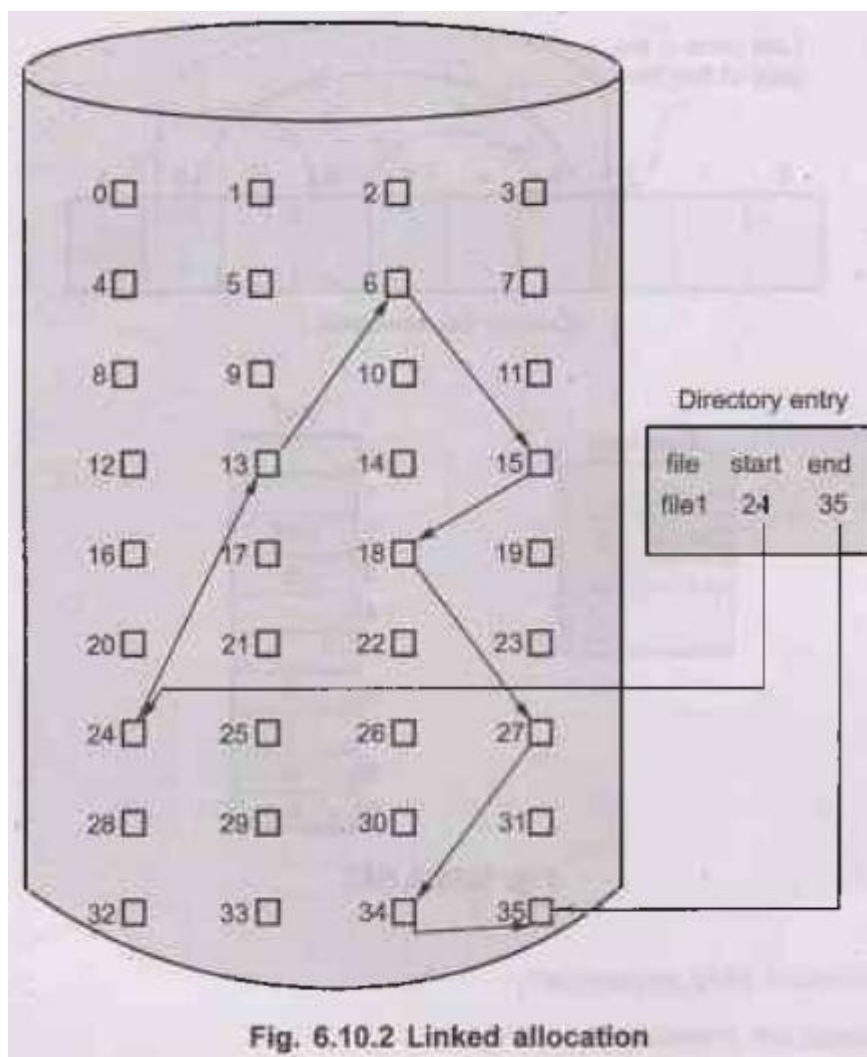
Disadvantages:

1. It suffers from external fragmentation.
2. Pre-allocation is required.

Linked Allocation

- Linked allocation is also called chained allocation. Operating system keeps an ordered list of free blocks. File descriptor stores pointers to the first block and each block stores pointer to the next block.
- Fig. 6.10.2 shows linked allocation. The disk blocks may be scattered anywhere on the disk. The directory contains a pointer to the first and last blocks of the file. No space is lost to disk fragmentation.

- Creation of new file is easy. For new file, simply create new entry in the directory. Reading a file is straightforward. User simply reads blocks by following pointers from block to block. There is no external fragmentation with linked allocation.
- To write to file, system finds a free block, and this new block is written to and linked to the end of the file.
- While creating a new file, it is not necessary to declare the size of the file. A file can grow as long as free blocks are available.
- Compaction can be used so that blocks of one file are located continuously on the disk. It optimizes disk access.
- File allocation table is an extension of the linked allocation method. Instead of putting the pointers in the file, keep a table of the pointers around. This pointer table can be quickly searched to find any random block in the file.



- Fig. 6.10.3 shows the file allocation table. All blocks on the disk must be included in the table. This method is used by windows operating system.

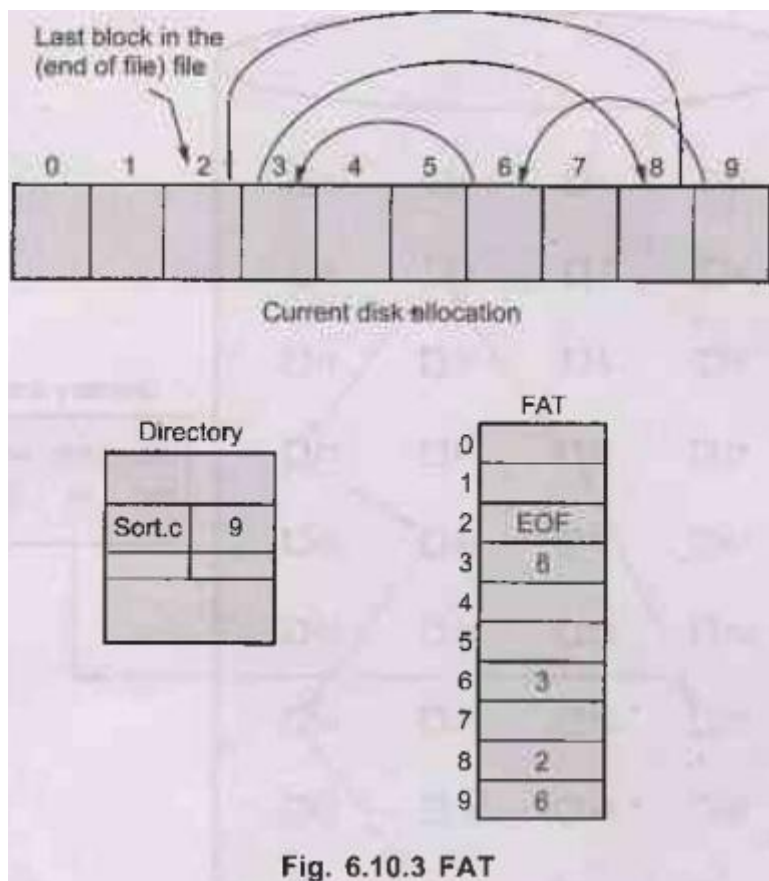
(Refer Fig. 6.10.3 on next page)

Characteristics

1. It supports fixed size portions.
2. Pre-allocation is possible.
3. File allocation table is one entry for a file.
4. Allocation frequency is low to high.

Advantages:

1. There is no external fragmentation.
2. It is never necessary to compact disk space.
3. Pre-allocation is not required.



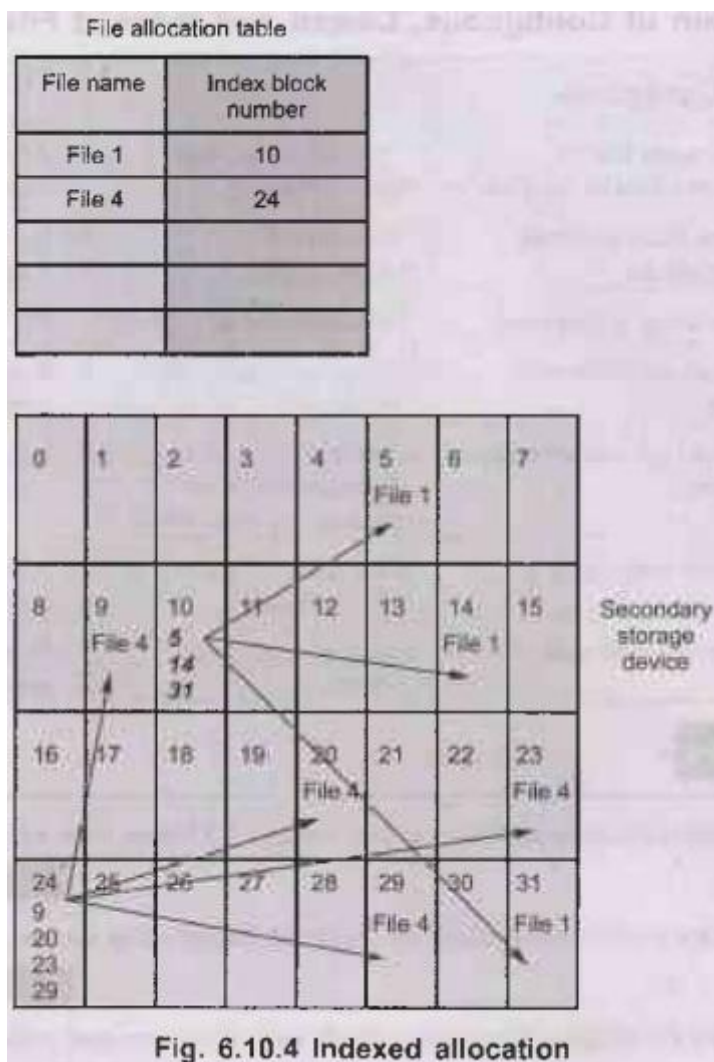
Disadvantages:

1. Files are accessed only sequentially.
2. Space required for pointers.
3. Reliability is not good.

4. Can not support direct access.

Indexed Allocation

- Indexed allocation method solves the problem of both contiguous and linked allocation. It uses concept of index block. Index block stores the entire pointer in one location. But the index block will occupy some space and thus could be considered as an overhead of the method.
- OS keeps a list of free blocks. It allocates an array to hold pointers to all the blocks used by the file. It allocates blocks only on demand. Fig. 6.10.4 shows indexed allocation.
- In indexed allocation, each file maintains its own index block. It contains an array of disk sector of addresses. For example: The n^{th} entry in the index block points to the n^{th} sector of the file. The directory contains the address of the index block of a file. To read the n^{th} sector of the file, the pointer in the n^{th} index block entry



- It supports direct access and without suffering from external fragmentation. Any free block anywhere on the disk may satisfy a request for more space.

- Indexed allocation does suffer from wasted space. The pointer overhead of the index block is generally greater than the pointer overhead of linked allocation.

Advantages:

1. It supports sequential and direct access.
2. No external fragmentation.
3. Faster than other two methods.
4. It supports fixed and variable size blocks.

Disadvantages:

1. Indexed allocation does suffer wasted space.
2. Pointer overhead is generally greater.

Comparison of Contiguous, Linked and Indexed File Allocation

Sr. No.	Contiguous	linked	Indexed
1.	Allocate each file to contiguous blocks on disk.	Allocate linked-list of fixed-sized blocks.	Allocate fixed-sized blocks for each file.
2.	It suffers from external fragmentation.	No external fragmentation.	No external fragmentation.
3.	Pre-allocation is required	Pre-allocation is possible.	Pre-allocation is possible.
4.	It support variable size portions.	It support fixed size portions.	It supports fixed and variable size portions.
5.	Simple to calculate random addresses.	Cannot calculate random addresses without reading previous blocks.	Supports random access.
6.	Allocation frequency is once.	Allocation frequency is low to high.	Allocation frequency is medium.
7.	No pointer overhead.	Space required for pointer.	Pointer overhead is generally greater.

Free Space Management

- Space that is allocated to files must be managed and space which is not allocated to any file must be managed. To keep track of free disk space, the system maintains a free space list. File allocation table and disk allocation both are required to manage free space properly.
- When a file is deleted, its disk space is added to the free-space list.
- Following are the techniques used for free space management:
 1. Bit tables or bit vector
 2. Chained free portions or linked list

3. Indexing or grouping

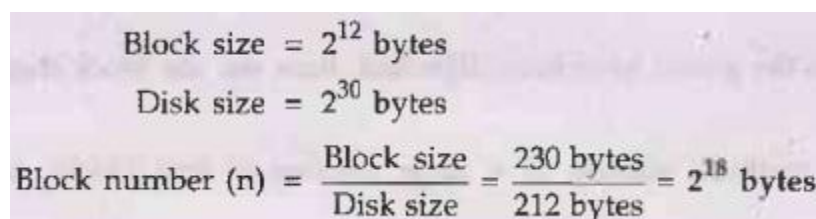
4. Free block list or counting

1. Bit tables or bit vector old

- Each block on the disk is represented by bit. It uses vector chain for blocks. Free block is represented by 0 and used block represented by 1.
- For example, consider a disk where blocks 3, 4, 5, 7, 9, 14, 15, 19, 30, 31, 32, 35, 36, 37, 38 are free blocks and rest of the blocks are allocated. The free space is shown below:

11100010101111001110111111111000110000

- The memory required for a block bitmap = Disk size / 8 × (block size of file system)
- Bit map requires extra space. For example:


$$\begin{aligned}\text{Block size} &= 2^{12} \text{ bytes} \\ \text{Disk size} &= 2^{30} \text{ bytes} \\ \text{Block number (n)} &= \frac{\text{Block size}}{\text{Disk size}} = \frac{2^{12} \text{ bytes}}{2^{30} \text{ bytes}} = 2^{-18} \text{ bytes}\end{aligned}$$

- When bit table is stored into the memory, then exhaustive search of the table can be slow and file system performance. Most of the file system use auxiliary data structure for bit table. File system also maintain summary table. Summary table contains sub-range, number of free blocks and maximum sized contiguous number of free blocks.
- Summary table is used to store information about contiguous free blocks. Whenever file system needs a number of contiguous blocks, it can scan the summary table to find an appropriate sub-range and then search that sub-range. This method is used in Apple Macintosh.

Advantage :

- a. Easy to find a free blocks
- b. It is as small as possible.

Disadvantage :

It may not be feasible to keep the bitmap in memory for large disks.

2. Link list

- In this method, all free space disk blocks are linked, keeping a pointer to the first free block. All file allocation methods used link list free space techniques.

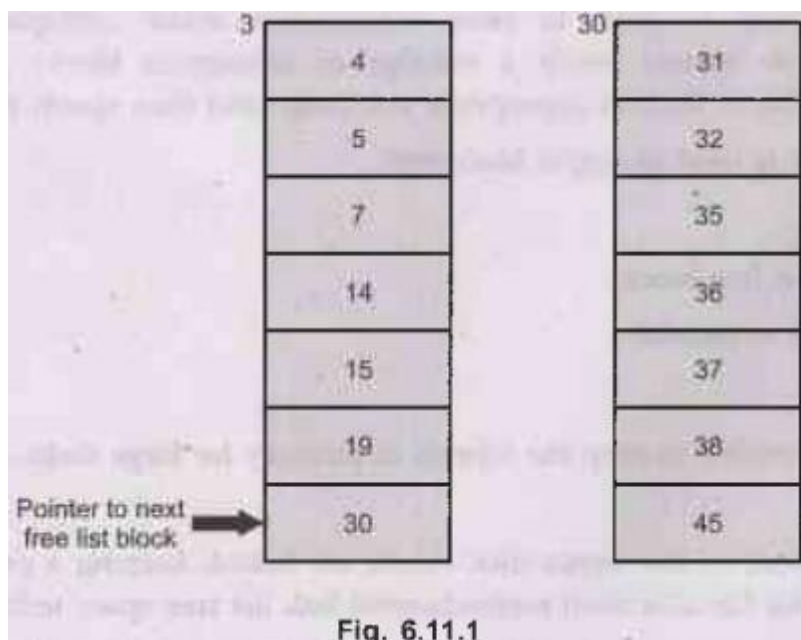
- There is small space overhead because there is no need for a disk allocation table. In the above example, free blocks are 3, 4, 5, 7, 9, 14, 15, 19, 30, 31, 32, 35, 36, 37, 38. Here free block pointer is on block number 3. Block 3 contains pointer to block 4, block 4 contains pointer to block 5 and block 5 contains pointer to block 7 and so on.
- This method is not efficient because to reach free block 9, we have to traverse 973 block 3, 4, 5 and 7 then we will reach to block 9.
- Disk will become quite fragmented after some use. Many portions will be a single block long.

3. Grouping

- First free block contains the addresses of n free blocks. The first n-1 of these blocks is actually free. The last block also contains the address of another n free block.
- Consider the free blocks: 3, 4, 5, 7, 9, 14, 15, 19, 30, 31, 32, 35, 36, 37, 38
- When all the blocks in the group have been allocated, then use the block that held the pointer.
- Because of grouping method, address of a large number of free blocks can be found quickly.

4. Counting

- It keeps the address of the first free block and the number n of free contiguous blocks that follow the first block. Each entry in the free space list then consists of a disk address and a count.

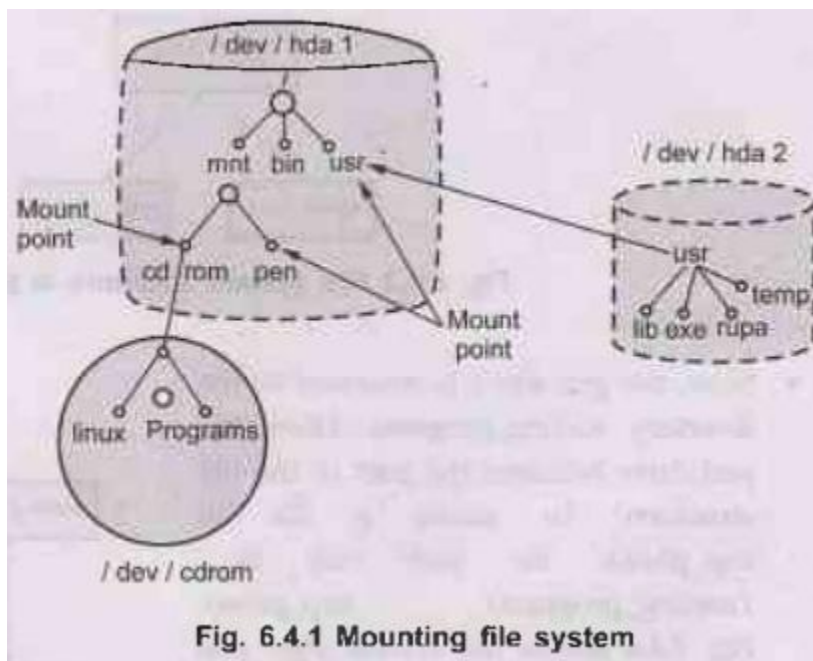


File System Mounting

- Files contained in a file system can be accessed only when the file system is mounted. Each device consists of its own local file system. Mounting is the operation that administrator/user perform to cause the file system on a physical storage device to appear as part of the file system.
- Mount system call is used on UNIX/Linux OS for mounting file system and unmount system call for disconnecting the file system. Mounting file system is used when there are more than one file system is used. Required file system is mounted in the host file system machine as a root directory.
- Some operating system support the mounting the file system at any point.
- File system stores directories and files. Each file system contains exactly one directory at the very top level, called the root directory for that file system. This root directory can contain other directories and files. One file system is designated the root file system or /. Every other file system is mounted under the root file system.

Procedure for mounting file system :

- To mount the file system, device name and location within the file structure



- Fig. 6.4.1 shows the mounting a file system on secondary storage device. File system manage mounted directories by using mount table. User can create soft links to files in mounted file system. Mount tables keep track of the actual physical location of the files.
- When a file system is mounted, the client can reference files by the local path name. There is no reference to remote host location, although files remain physically located at the remote site.

Busy file system is can not unmount by using unmount command. A file system is considered busy in following situation :

1. File system is shared in between the users.
2. A program has a file open in that file system.
3. A user is accessing a file/directory in the file system.

Example: Consider a file structure of the pen drive. Following is Fig. 6.4.2 of file

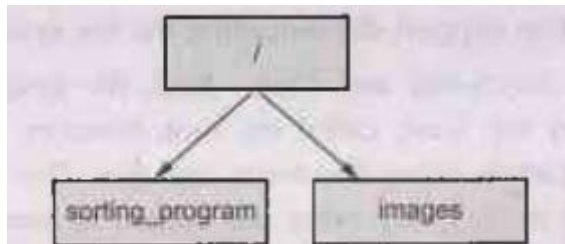


Fig. 6.4.2 File system before mounting

- The file system structure in the pen drive is as follows:

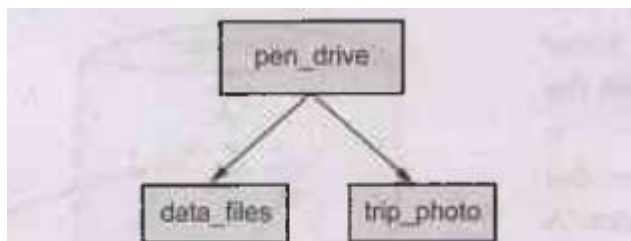


Fig. 6.4.3 File system structure in pen drive

- Now, the pen drive is mounted to the directory sorting_program. Here, the pen drive becomes the part of the file

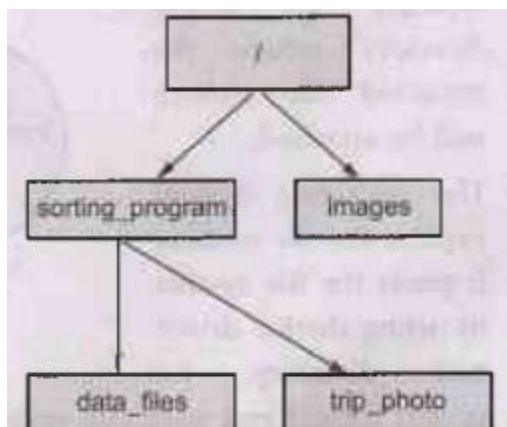


Fig. 6.4.4 File system after mounting

Advantages of mounting:

1. It is a user-oriented naming scheme

2. It makes sense to be able to mount at any level.

Disadvantages of mounting :

1. The hardware is less obvious since devices can be located anywhere in the tree structure

Example 6.4.1 Discuss the advantages and disadvantages of supporting links to files that cross mount points.

Solution :

Advantages:

1. Greater transparency
2. User does not need to be aware of mount points.

Disadvantages:

1. The error condition would expose to the user that a link is a dead link and that the link does indeed cross file system boundaries.
2. The file system containing the link might be mounted while the file system containing the target file might not be.

Partitions and Mounting

A **partition** is a logically divided section of a physical storage device (like HDD, SSD) that the operating system treats as a separate unit.

Key Points:

- **Purpose:** Organize data, run multiple OSes, improve performance, and manage backups.
- **Types of Partitions:**
 1. **Primary Partition** – Can contain an OS; maximum of 4 in MBR partitioning.
 2. **Extended Partition** – Special partition that can hold multiple **logical partitions**.
 3. **Logical Partition** – Exists inside an extended partition.
- **Partition Table:** Stores details about partitions (start sector, size, type).

Example: If you have a 1TB disk:

- Partition 1: 200GB → Windows OS
- Partition 2: 300GB → Linux OS
- Partition 3: 500GB → Data storage

2. Mounting in OS

Mounting is the process of making a file system accessible to the OS by attaching it to a **mount point** in the directory tree.

How it Works:

- A **mount point** is a directory where the contents of the file system will appear.
- Once mounted, files and directories from that storage device become part of the main file system hierarchy.

Example in Linux:

Bash

Copy code

```
sudo mount /dev/sdb1 /mnt/data
```

- /dev/sdb1 → Device/partition to mount
- /mnt/data → Mount point directory

After mounting, /mnt/data will show the files from /dev/sdb1.

Mounting Types:

1. **Manual Mounting** – User runs the mount command.
2. **Automatic Mounting** – Defined in /etc/fstab so it mounts at boot.
3. **Hot Mounting** – Mounting removable devices (USB, CD) without reboot.

Unmounting:

To safely remove a mounted file system:

Bash

Copy code

```
sudo umount /mnt/data
```

This ensures all pending writes are completed.

3. Relationship Between Partitions and Mounting

- **Partition:** Physical/logical division of storage.
- **Mounting:** Linking a partition's file system into the OS directory tree.
- A partition must be **formatted** with a file system (e.g., ext4, NTFS, FAT32) before it can be mounted.

File Sharing

- Multiprogramming and multitasking operating system support the multiple user. These users perform various operations on the files. Files are shared between numbers of users. File protection, naming and sharing is issue for file sharing.
- Directory structure may be allows files to be shared by users. Sharing must be done through a protection scheme.
- In single user system, operating system maintains attributes for all files and directory. According to the attributes, operating system allows to operate the files for users. But on the multi-user system, more attributes are needed. An attributes like owner, group and other user are required. These are the access permission for files and directory.
- Access permissions are of three types : Read, Write, and execute.
 1. An "r" indicates read permissions for a file or directory.
 2. A "w" indicates write permissions.
 3. An "x" indicates execution permissions if it is a file and search permissions if it is a directory.
 4. Owner is the file owner who created a file. Owner having all rights (read, write and execute).
 5. The groups of other user have some limited access permission. Mostly read and execute permission.
 6. Last categories of user have only execute permission.
- Example In UNIX operating system, file owner can perform all operation on a file. Group member can execute one subset of those operations and other than the group user can execute another subset of operations.
- When a user requests an operation on a file, the user ID can be compared with the owner attribute to determine if the requesting user is the owner of the file. Likewise, the group IDs can be compared.

Remote file systems

- Computer network is used to support remote file system. User can access remote file system. File Transfer Protocol (FTP) and Secure Shell protocol (SSH). Other supporting protocol like distributed file system and world wide web is also used for remote file system.
- SSH is a protocol for secure remote access to a machine over untrusted networks.
- A client-server model has three components:

1. Service: A service is a software entity that runs on one or more machines. It provides an abstraction of a set of well-defined operations in response to applications' requests.

2. Server: A server is an instance of a particular service running on a single machine.

3. Client: A client is a software entity that exploits services provided by servers.

- Client application program running on the local machine requests a service from another application program, server running on the remote machine. Commonly server provides service to any client, not a particular client.
- Generally, a client application program that requests a service should run only when it is needed. A server program providing service should run all the time, as it does not know when its services will be needed.
- Client-server model allows clients to mount remote file systems from servers. The server is connected with multiple clients and it serve to all clients. Client and user-on-client identification is insecure or complicated.
- A client opens the communication channel using IP address of the remote host and the port address of the specific server program running on the host i.e. active open. Request-response may be repeated several times, the process is finite. The client closes the communication channel with an active close.
- A server program opens its door for incoming requests from clients but never initiates a service unless explicitly requested i.e. passive open. A server program is infinite and runs unless a problem occurs.
- Two or more clients can run at the same time on a machine is called concurrency in client. One client must start, run and terminate before another client may start. It is called iterative.
- Network File System (NFS) is standard for UNIX client-server file sharing protocol and Common Internet File System (CIFS) is standard Windows operating system protocol. The NFS client maintains a cache of file and directory attributes. The default settings will not ensure that files created or modified on one system will be visible on another system within a minute of file creation or modification. Network file system is common distributed file sharing system.
- File data modifications might not be visible on any NFS client system other than the one where the modifications are being made until an NFS commit is executed. Most NFS clients will issue a commit as part of closing a file. If multiple systems are reading files that might be modified, file system locking should be used.
- NFS communication protocols lets processes running in different environments share a file system. NFS implements a file system model that is almost identical to a UNIX system.

- Distributed Information Systems such as LDAP, DNS, NIS, and Active Directory implement unified access to information needed for remote computing. It is also known as distributed naming services.
- A name in a distributed system is a string of bits or characters used to refer to an entity. To resolve names a naming system is needed. Names are used to share resources, uniquely identify entities and refer to locations.
- The naming and locating facilities jointly form a naming system that provides the users with an abstraction of an object that hides the details of how and where an object is actually located in the network.
- It provides a further level of abstraction when dealing with object replicas. Email and FTP is used for sending information to remote users. But these two protocols will not support true file sharing. Domain Name System (DNS) is used to map the host name to network address translation for the internet. DNS replace the Email and FTP. The DNS is a hierarchical distributed naming system.
- On windows operating system, user name and password is used to create login on the server. Using this authentication method, server will allow and deny access to a requested file system. CIFS is used in conjunction with login name and password.
- Industry is moving towards the new technology. So new protocol called 18. Lightweight Directory Access Protocol (LDAP) is used for secure distributed naming service. LDAP allows user to search for an individual without knowing where they are located.
- An LDAP directory can be distributed among many servers. Each server can have a replicated version of the total directory that is synchronized periodically. An LDAP server is called a Directory System Agent.
- An LDAP server that receives a request from a user takes responsibility for the request, passing it to other DSAS as necessary, but ensuring a single coordinated response for the user.

Failure modes

Reasons for local file system failures:

1. Media fails where file system is stored
2. Corruption of directory structure
3. Power cable and hard disk cable failure
4. Disk controller failure
5. Host adapter failure.

- Apart from these reasons, there are many other reasons for remote file systems. Bandwidth and communication also cause the delay.

Consistency semantics

- Consistency semantics is related with file sharing on the network. When does the changes of the original file is reflected to other users. If there is difference in the contents of the file, then it creates the problem.
- It deals with the consistency between the views of shared files on a networked system. When one user changes the file, when do other users see the changes of the content?
- Define when modifications of the file data made by a user are observable by other users

1. Unix semantics

2. Session Semantics

3. Immutable shared-files semantics

4. Transaction-like semantics

- Difference operating systems consistency semantics is discussed here :

1. UNIX semantics

- Unix file system (UFS) implements:
 - a. Writes to an open file visible immediately to other users of the same open file.
 - b. Sharing file pointer to allow multiple users to read and write concurrently
- In UNIX semantics, a file is coupled with a single physical image that is associated as special resource. If there is conflict for single then it causes delays in user processes.
- Centralized system uses UNIX semantics. This is common for single processor systems, but difficult to achieve for distributed file systems.

2. Session semantics

Andrew File System (AFS) implemented complex remote file sharing semantics.

1. Writes to a file by a user is not visible to other users.

2. Once the file is closed, the changes are visible only to new sessions.

- In this semantics, a file can be associated with multiple views. Almost no constraints are imposed on scheduling accesses. No user is delayed in reading or writing their personal copy of the file.
- AFS file systems may be accessible by systems around the world. Access control is maintained through complicated access control lists, which may grant access to the entire

world or to specifically named users accessing the files from specifically named remote environments.

- Questions arise about how to handle reads/writes by multiple processes. The file-level transfer model should be used.

3. Immutable-shared-files semantics

- Under this system, when a file is declared as shared by its creator, it becomes immutable and the name cannot be re-used for any other resource. Hence it becomes read-only and shared access is simple.
- Once a file is declared as shared by its creator, it cannot be modified.
- An immutable file has two key properties. Its name may not be reused and its contents may not be altered.

SYSTEM PROTECTION

GOALS OF PROTECTION

- Operating system consists of a collection of objects, hardware or software. Each object has a unique name and can be accessed through a well-defined set of operations.
- Protection problem - ensure that each object is accessed correctly and only by those processes that are allowed to do so.
 1. Obviously to prevent malicious misuse of the system by users or programs.
 2. To ensure that each shared resource is used only in accordance with system *policies*, which may be set either by system designers or by system administrators.
 3. To ensure that errant programs cause the minimal amount of damage possible.

Note:

- Protection systems only provide the *mechanisms* for enforcing policies and ensuring reliable systems. It is up to administrators and users to implement those mechanisms effectively.

PRINCIPLES OF PROTECTION

- Programs, users and systems should be given just enough privileges to perform their tasks-
- 1. The *principle of least privilege* dictates that programs, users, and systems should be given just enough privileges to perform their tasks. This ensures that failures do the least amount of harm and allow the least of harm to be done.
 - For example, if a program needs special privileges to perform a task, it is better to make it a SGID program with group ownership of "network" or "backup" or some other pseudo group, rather than SUID with root ownership. This limits the amount of damage that can occur if something goes wrong.
- 2. Typically each user is given their own account, and has only enough privilege to modify their own files. The root account should not be used for normal day to day activities – The System Administrator should also have an ordinary account, and reserve use of the root account for only those tasks which need the root privileges.

DOMAIN OF PROTECTION

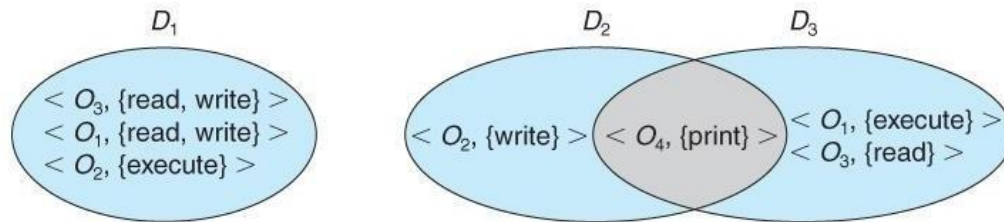
- A computer can be viewed as a collection of *processes* and *objects* (both Hardware & Software).
- The *need to know principle* states that a process should only have access to those objects it needs to accomplish its task, and furthermore only in the modes for which it needs access and only during the time frame when it needs access.
- The modes available for a particular object may depend upon its type.

Domain Structure

- A *protection domain* specifies the resources that a process may access.

- Each domain defines a set of objects and the types of operations that may be invoked on each object.
- An *access right* is the ability to execute an operation on an object.
- A domain is defined as a set of $\langle \text{object}, \{\text{access_right_set}\} \rangle$ pairs.

Example,



System with three protection domains

- The association between a process and a domain may be *static* or *dynamic*.
 - If the association is *static*, then the need-to-know principle requires a way of changing the contents of the domain dynamically.
 - If the association is *dynamic*, then there needs to be a mechanism for *domain switching*.
- Domains may be realized in different fashions - as users, or as processes, or as procedures.
 - As users – In this case, the set of objects that can be accessed depends on the identity of the user. Domain switching occurs when the user is changed (generally when one user logs out and another user logs in).
 - As process – In this case, the set of objects that can be accessed depends on the identity of the process. Domain switching occurs when one process sends a message to another process and then waits for a response.
 - As procedure – In this case, the set of objects that can be accessed corresponds to the local variables defined within the procedure. Domain switching occurs when a procedure call is made.

ACCESS MATRIX

- General model of protection can be viewed abstractly as a matrix, called as Access Matrix. The rows of the access matrix represent domains and the columns represent objects.
- Each entry in the matrix consists of a set of access rights.
- The entry $\text{access}(i, j)$ defines the set of operations that a process executing in domain D_i can invoke on object O_j .

Example 1,

Consider four domains D_1 through D_4 and four objects – three files F_1 through F_3 and one laser printer.

Here, a process executing in domain D_1 can read files F_1 and F_3 . A process executing in domain D_4 has the same privileges as one executing in domain D_1 , in addition it can also write onto

files F_1 and F_3 . The laser printer can be accessed only by a process executing in domain D_2 .

object domain	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

Access Matrix

- The access matrix scheme provides a mechanism for specifying a variety of policies. Policies of access matrix concerning protection involve which rights should be included in the $(i, j)^{\text{th}}$ entry.
- We must also decide the domain in which each process executes. This policy is usually decided by the operating system.
- The Users decide the contents of the access-matrix entries.
- Access matrix provides an appropriate mechanism for defining and implementing strict control for both static and dynamic association between domain and processes.
- When we switch a process from one domain to another, we execute an operation (switch) on an object (the domain).
- We can control domain switching by including domains among the objects of the access matrix.
- Processes should be able to switch from one domain (D_i) to another domain (D_j) if and only if a switch right is given to $access(i, j)$.

Example 2,

A process executing in domain D_2 can switch to domain D_3 or to domain D_4 . A process in domain D_4 can switch to D_1 and one in domain D_1 can switch to D_2 .

object domain	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch
D_3		read	execute					
D_4	read write		read write		switch			

Access matrix for the Example 1 with domains as objects

- Allowing controlled change in the contents of the access matrix entries requires three additional operations – *copy*, *owner* and *control*.
 - The ability to *copy* rights is denoted by an asterisk, indicating that processes in that domain have the right to copy that access within the same column, i.e. for the same object. There are two important variations:
 - If the asterisk is removed from the original access right, then the right

is *transferred*, rather than being copied. This may be termed a *transfer* right as opposed to a *copy* right.

- If only the right and not the asterisk are copied, then the access right is added to the new domain, but it may not be propagated further. That is the new domain does not also receive the right to copy the access. This may be termed a *limited copy* right.

object domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute		

(a)

object domain	F_1	F_2	F_3
D_1	execute		write*
D_2	execute	read*	execute
D_3	execute	read	

(b)

Example 3,

Access matrix with copy rights

- The *owner* right adds the privilege of adding new rights or removing existing ones.

Example 4,

object domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		read* owner	read* owner write
D_3	execute		

(a)

object domain	F_1	F_2	F_3
D_1	owner execute		write
D_2		owner read* write*	read* owner write
D_3		write	write

(b)

Access matrix with owner rights

- Copy and owner rights only allow the modification of rights within a column. The

addition of *control rights*, which only apply to domain objects, allow a process operating in one domain to affect the rights available in other domains.

Example 5,

A process operating in domain D2 has the right to control any of the rights in domain D4.

object domain	F_1	F_2	F_3	laser printer	D_1	D_2	D_3	D_4
D_1	read		read			switch		
D_2				print			switch	switch control
D_3		read	execute					
D_4	write		write		switch			

Modified access matrix of Example 2

IMPLEMENTATION OF ACCESS MATRIX

There are several methods of implementing the access matrix -

1. Global Table

- The simplest approach is one big global table with < domain, object, rights_set > entries.
- Unfortunately this table is very large (even if sparse) and so cannot be kept in memory (without invoking virtual memory techniques).
- There is also no good way to specify groupings - If everyone has access to some resource, then it still needs a separate entry for every domain.

2. Access Lists for Objects

- Each column of the table can be kept as a list of the access rights for that particular object, discarding blank entries.
- For efficiency a separate list of default access rights can also be kept, and checked first.

Example,

Access lists for objects for *Example 1*

Object $F_1 \rightarrow$ Read (R), Read Write (RW)

Object $F_2 \rightarrow$ Read (R)

Object $F_3 \rightarrow$ Read (R), Execute (X), Read Write (RW)

3. Capability Lists for Domains

- In a similar fashion, each row of the table can be kept as a list of the capabilities of that domain.
- Capability lists are associated with each domain, but not directly accessible by the domain or any user process.

- Capability lists are themselves protected resources, distinguished from other data in one of two ways:
 - A *tag*, possibly hardware implemented, distinguishing this special type of data.
 - The address space for a program may be split into multiple segments, at least one of which is inaccessible by the program itself, and used by the operating system for maintaining the process's access right capability list.

Example,

Capability lists for domains for *Example 1*

Domain $D_1 \rightarrow$ Read (R), Read (R)

Domain $D_2 \rightarrow$ Print

Domain $D_3 \rightarrow$ Read (R), Execute (X)

Domain $D_4 \rightarrow$ Read Write (RW), Read Write (RW)

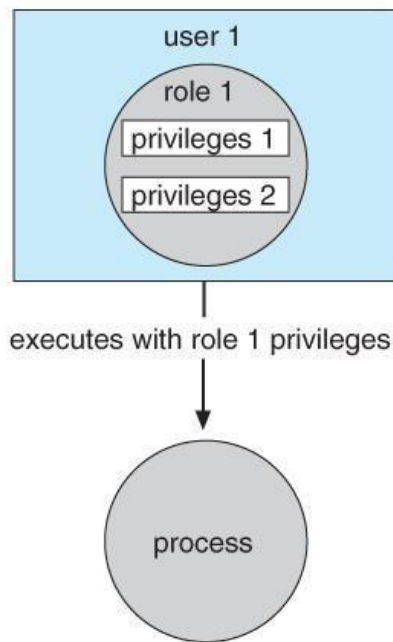
4. A Lock-Key Mechanism

- Each resource has a list of unique bit patterns, termed locks.
- Each domain has its own list of unique bit patterns, termed keys.
- Access is granted if one of the domain's keys fits one of the resource's locks.
- Again, a process is not allowed to modify its own keys.

ACCESS CONTROL

- Access control is a security technique that regulates who or what can view or use resources in a computing environment.
- Three types of access control are there –
 1. **Mandatory access control (MAC)** - This is a security model in which access rights are regulated by a central authority based on multiple levels of security.
 2. **Discretionary access control (DAC)** - This is an access control method in which owners or administrators of the protected system, data or resource set the policies defining who or what is authorized to access the resource.
 3. **Role-based access control (RBAC)** - This is a widely used access control mechanism that restricts access to computer resources based on individuals or groups with defined business functions.
 - The role-based security model relies on a complex structure of role assignments, role authorizations and role permissions developed using role engineering to regulate employee access to systems.
 - RBAC systems can be used to enforce MAC and DAC frameworks.
 - RBAC supports the principle of least privilege, and reduces the susceptibility to abuse as opposed to SUID or SGID programs.

Example for RBAC,



Role-based access control in Solaris 10

REVOCATION OF ACCESS RIGHTS

- The need to revoke access rights dynamically raises several questions:
 - *Immediate versus delayed* - If delayed, can we determine when the revocation will take place?
 - *Selective versus general* - Does revocation of an access right to an object affect *all* users who have that right, or only some users?
 - *Partial versus total* - Can a subset of rights for an object be revoked, or are all rights revoked at once?
 - *Temporary versus permanent* - If rights are revoked, is there a mechanism for processes to re-acquire some or all of the revoked rights?
- With an access list scheme revocation is easy, immediate, and can be selective, general, partial, total, temporary, or permanent, as desired.
- With capabilities lists the problem is more complicated, because access rights are distributed throughout the system. A few schemes that have been developed include:
 - *Reacquisition* - Capabilities are periodically revoked from each domain, which must then re-acquire them.
 - *Back-pointers* - A list of pointers is maintained from each object to each capability which is held for that object.
 - *Indirection* - Capabilities point to an entry in a global table rather than to the object. Access rights can be revoked by changing or invalidating the table entry, which may affect multiple processes, which must then re-acquire access rights to continue.
 - *Keys* - A unique bit pattern is associated with each capability when created, which can be neither inspected nor modified by the process.

- A master key is associated with each object.
- When a capability is created, its key is set to the object's master key.
- As long as the capability's key matches the object's key, then the capabilities remain valid.
- The object master key can be changed with the set-key command, thereby invalidating all current capabilities.
- More flexibility can be added to this scheme by implementing a *list* of keys for each object, possibly in a global table.

Short Answer Questions

1. What is a file?
2. List any four file attributes.
3. What is a file operation? Name any two basic file operations.
4. Define sequential access method.
5. Define direct access (relative access) method.
6. What is a directory?
7. What is the purpose of a file system?
8. What is a single-level directory?
9. What is a two-level directory?
10. What is file mounting?
11. What is contiguous allocation?
12. What is linked allocation?
13. What is indexed allocation?
14. Define protection and state its primary goal.
15. What is an access matrix in operating systems?